Using Reducers

Global data have different problems than local data. In particular, we need to make sure that changes to the global data store happen consistently for all users so that different parts of the application all have access to data in the same state.

We're going to solve our global data problem by refactoring our data using a JavaScript pattern called a *reducer* and a related structure called a *store*.

A *reducer* is a JavaScript function that takes two arguments. The first argument is an object representing a state. The second argument represents an action to take. The return value is a new state object representing the state of the data after the action is performed. The action is meant to describe the state change in terms of application logic, while the reducer converts that to changes in data.

Let's look at a simple example in which we will count American coins. The state of the world is the total number of coins and the total value of the coins. A reducer to partially handle this task might look like this:

```
const initialState = {count: 0, value: 0}

const reducer = (state, action) {
  switch (action.type) {
    case "AddPenny": {
      return { count: state.count + 1, value: state.value + 1 }
    }
    case "AddNickel": {
      return { count: state.count + 1, value: state.value + 5 }
    }
    // and so on...
  }
}
```

Then you'd call this method with something like this:

```
const withAPenny = reducer(initialState, {type: "AddPenny"})
const pennyAndNickel = reducer(withAPenny, {type: "AddNickel"})
```

The idea is that you only change state via the reducer function, which—